

Nils MOREAU--THOMAS - Yanis WONG

Sommaire

- [Sommaire](#)
- [Lancement](#)
 - [Avec les extensions :](#)
 - [Avec la ligne de commande :](#)
- [Structure](#)
 - [Explications sur la structure](#)
- [Informations utiles](#)
 - [Navigation](#)
 - [Documents et branches](#)
 - [Guide des touches](#)
 - [Listes des tâches](#)
 - [Documentations du code](#)
 - [Fichiers Texture](#)
 - [Fichiers Lighting](#)
 - [Dossier Draw](#)
- [Rapport](#)
 - [Bugs et problèmes résolus](#)
 - [Identification des bugs](#)
 - [Problème d'optimisation du moteur de rendu](#)
 - [Choix techniques](#)
 - [Refactorisation des coordonnées](#)
 - [Informations ajoutées au JSON](#)
 - [Difficultés](#)
 - [Problèmes d'équipe](#)
 - [Problèmes du sujet](#)
 - [Conclusion](#)

Lancement

Avec les extensions :

Avec [CMake](#) et [CMake Tools](#) il suffit d'exécuter la cible `[main]`.

Avec la ligne de commande :

Depuis la racine faire :

```
mkdir -p build && cd build
cmake .. -G "MinGW Makefiles"
cd .. && cmake --build ./build --target all --config Debug -j 16
cd bin && ./main.exe fichier/a/lire.json
```

Exemple : `./main.exe ../data/path_1.json`.

 : Les commandes précédant l'exécution peuvent ne pas marcher ou être différentes selon l'ordinateur.

Structure

```
├── assets : shaders & textures
├── data : fichiers JSON de rails
│   └── DATADESC.md : description des données JSON
├── docs : documents et diagrammes
├── lib : librairies
├── src : code source
│   ├── camera : gestion des caméras et des entrées utilisateur
│   ├── lighting : gestion de l'éclairage (placement des lights)
│   ├── global : éléments partagés
│   ├── texture : utilitaires de chargement et de gestion de textures
│   ├── draw : rendu des objets 3D (rails, train, gare, arbres, scène)
│   │   └── shared.cpp : helpers et constantes partagées entre les dessins
│   ├── main.cpp : point d'entrée, initialisation et boucle principale
│   └── json_data.hpp : structures de sérialisation JSON
└── README.md : ce fichier
```

Explications sur la structure

Le projet est organisé autour de trois zones principales :

- `assets/` : les ressources, notamment les shaders et les textures
- `data/` : fichiers JSON de parcours et `DATADESC.md` qui décrit la structure des données
- `docs/` : documents et diagrammes
- `lib/` : librairies externes (GLFW, glad, glbasimac, nlohmann)
- `src/` : code source

La structure est pensée pour séparer :

- le code de rendu (`src/draw`),
- les utilitaires (`src/camera`, `src/lighting`, `src/texture`, `src/global`),
- et les librairies (`lib/`).

Cela rend plus facile l'évolution du projet (et sa maintenance)

Le projet initialisé sur la base du TD04 de synthèse d'image, et remaniée de manière archaïque au niveau des `CMakeLists.txt` et de la structure des librairies

Informations utiles

Navigation

Il y a une extension pratique pour naviguer dans des endroits du code sur `VS Code` :

`Todo Tree`

Avec `NOTE` : d'ajoutée dans le paramètre de l'extension `todo-tree.general.tags`.

Il est aussi recommandé d'utiliser l'`Outline` dans le panneau de droite de `VS Code`.

Documents et branches

Il y a un document `DATADESC.md` qui décrit les fichiers JSON et leurs visualisations.

Il y a la branche `exp-nils` qui contient des tests de morceaux de codes réalisés par Nils, il contient actuellement presque toutes les tâches sauf les deux bonus

Guide des touches

Touche	Action	Note
Esc	Quitter l'application	
Z Q S D	Déplacer/Bouger la caméra	
Molette souris	Avancer/Reculer la caméra	Hors mode FPS
Déplacement souris	Tourner la caméra	Uniquement mode FPS
R	Changer le mode de rendu (fil de fer ou plein)	
F	Changer le mode d'éclairage (flat ou phong)	
C	Changer le mode de la camera (orbitale, top, fps)	
G	Afficher/Masquer la grille	

Listes des tâches

✓ = Bonus énuméré dans l'énoncé 🌊 = Bonus supplémentaire ✅ = Fait 🚧 = En cours ❌ = À faire

Status	Catégorie	En +	Description	Commentaire	Qui
✅	📁 Structure		Créer la structure du projet	Reprendre la base du TD04	👤 Nils
✅	🔧 Modélisation		Faire les Rails	Faire Courbé et Droit	👤 Yanis

Status	Catégorie	En +	Description	Commentaire	Qui
✓	✂ Modélisation		Faire le Train	+ Faire triangle et cylindre fermé	● Nils
✓	✂ Modélisation		Faire la Gare		● Nils
✓	📄 JSON		Lire le JSON (JsonData)	Serialisé en JsonData	● Nils
🚧	📄 JSON		Créer le parcours	Depuis JsonData	● Yanis
✓	📄 JSON		Placer la gare		● Nils
✗	📄 JSON		Placer le train		● Yanis
✓	📄 JSON		Lancement avec JSON en argument		● Nils
✓	💻 IHM		Touche pour quitter	Touche Échap	● Nils
✓	💻 IHM		Touche pour toggle modes d'éclairages	Touche F	● Nils
✓	💻 IHM		Caméra FPS	ZQSD to move where facing, Pan to rotate	● Nils
✓	💻 IHM		Caméra ORBITAL	ZQSD to rotate, Scroll to zoom	● Nils
✓	👁 Rendu		Éclairage "flat"	Par défaut sur OpenGL	● OpenGL
✓	👁 Rendu		Éclairage "réaliste" *	Un soleil et phares du train	● Yanis
✓	📁 Structure	🌊	Refactorisation des coordonnées	Voir Informations supplémentaires	● Nils
✓	📁 Structure	🌊	Correction des bugs	Voir Informations supplémentaires	● Nils
✓	📁 Structure	🌊	Optimisation du moteur de rendu	Voir Informations supplémentaires	● Nils
✓	💻 IHM	🌊	Touche pour toggle modes de rendu	Touche R	● Nils
✓	💻 IHM	🌊	Caméra TOP	ZQSD to move (X,Z), Scroll to zoom (Y)	● Nils
✓	✂ Modélisation	🌊	Grille des cellules de rails	Pour visualiser le placement	● Nils
✓	✂ Modélisation	🌊	Primitives supplémentaires	Triangle rectangle et cylindre fermé	● Nils

Status	Catégorie	En +	Description	Commentaire	Qui
✓	✂ Modélisation	✓	Modéliser (+ placer) du décors	Genre un arbre	● Nils
✓	📄 JSON	✓	Ajouter des trucs dans le JSON	Placement d'arbres	● Nils
✗	👁 Rendu	✓	Animer le train	Ptet avec des fonctions dans les rails	-
✗	👁 Rendu	✓	Ajouter des lumières	Genre à la gare	-

* = D'après le PDF : "vous devrez positionner une lumière directionnelle (orientée selon votre choix mais qui éclaire la scène comme un soleil), et une source de lumière ponctuelle au niveau de l'avant de votre train. Par ailleurs au moins un élément de votre application devra être texturé !"

Temps passé par Nils

- Structure : 12h
- Modélisation du train : 3h30
- Création des primitives + utilisation sur le train : 5h
- Lecture du JSON + en ligne de commande : 2h
- Grille : 1h
- Cameras : 3h
- Refactorisation des coordonnées : 4h30
- Correction de bugs du moteur de rendu : 2h
- Optimisation du moteur de rendu : 1h
- Ajout des arbres : 1h30
- Ajout du helper des textures : 4h
- Ajout du helper des lumières : 4h
- Expérimentation sur la visualisation du parcours avec les rails (branche [exp-nils](#)) : 2h

Total : 45h30

Documentations du code

Fichiers Texture

Dans [texture.cpp](#) il y a :

- La structure `ImageData` qui stocke des informations sur une image chargée
- Les listes `loadedTextures` et `loadedImages` stockant ce qui est chargés
- Les fonctions `loadImage()` et `createTextureFromImage()` servant à charger une image et en faire une texture
- Les fonctions `startTextureRender()` et `endTextureRender()` pour charger les textures
- La fonction `freeAllResources()` qui libère la mémoire pour tout ce qui a été chargé (ce qui justifie l'existence de tout ça)

Fichiers Lighting

Dans `lighting.cpp` il y a :

- La fonction `handleMainLighting(double time)` qui crée le "soleil"
- La fonction `handleTrainLighting(float x, float y, float z)` qui crée aux coordonnées données le phare du train

Dossier Draw

Draw contient des fichiers de rendus intitulées par le nom de ce qui est dessiné Le tout est appelé dans `draw_scene.cpp`

Dans `shared.cpp` il y a des fonctions utilitaires:

- `drawShapeWithColor(shape, color)` ou `drawShapeWithColor(shape, r, g, b)` pour dessiner une forme avec un couleur (un vecteur ou 3 valeurs)
- `moveOrigin(x, y, z)`, `rotateOrigin(angle, x, y, z)` et `scaleOrigin(x, y, z)` pour bouger, tourner et faire une homotétié sur le "curseur de dessin" (ç.à.d là ou on dessine une forme, comme un crayon ou un curseur turtle en Python)

Rapport

Bugs et problèmes résolus

Identification des bugs

Deux accès à des `std::vector` dans `glbasimac/tools/mesh.hpp` faisait crash le programme car il y avait un accès à des données inexistantes sur les ID de VBO (par le temps que la frame prend à s'exécuter) lors de la "vidange" du Buffer

```
Qui causait l'erreur [...] std::vector<Tp, _Alloc>::size_type = long unsigned int]:  
Assertion '__n < this->size()' failed. et parfois sans s'afficher dans la console.
```

Les chemins pour les shaders étaient en relatif (depuis le dossier `bin`) ce qui contraignait l'exécution depuis EXCLUSIVEMENT `bin`, si l'on exécutait le `.exe` sans être placé dans le dossier `bin` cela donnais l'erreur suivante :
`ERROR GL : erreur dans le fichier [...] \src\main.cpp à la ligne 276 : INVALID_VALUE (A numeric argument is out of range).`

Nous avons donc modifié `glbi_engine.cpp` pour avoir une variable de chemin pointant vers `assets/` qui se met à jour depuis le `main.cpp`.

```
Avec assetsPath = (fs::path(argv[0]).parent_path() / "../assets/").string(); qui :
```

1. Converti l'argument 0 (chemin du `.exe`) en objet `fs::path`
2. Récupère son dossier parent (`bin/`)
3. Va vers `../assets`

```
Merci la documentation de filesystem.
```

Problème d'optimisation du moteur de rendu

Les géométries et les points étaient initialisés de 0 dans `glbi_convex_2D_shape.cpp` et `glbi_set_of_points.cpp` ce qui faisait éventuellement crash et causait de la latence dans le programme. Nous avons donc ajouté des `.reInit()` dans ces fichiers pour éviter cela.

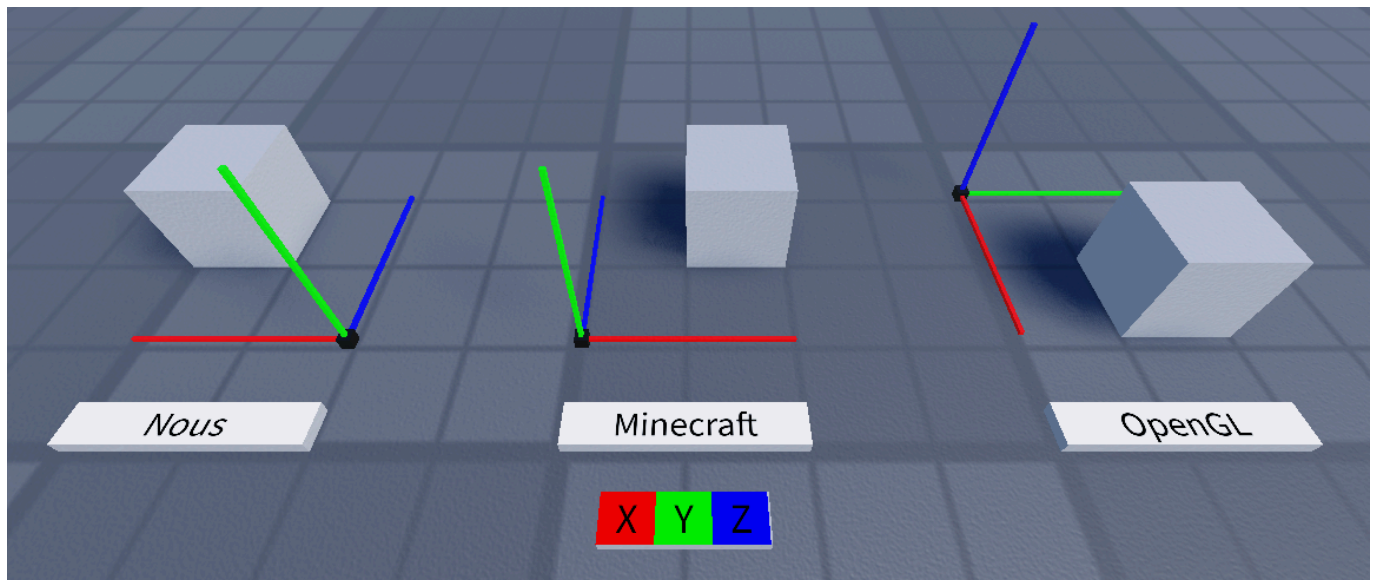
Choix techniques

Refactorisation des coordonnées

Le système de coordonnées a été refactorisé pour mieux s'aligner avec les conventions OpenGL.

L'ancien repère **Right-handed Z-up** (X+ derrière, Y+ droite, Z+ haut) a été remplacé par un repère **Right-handed Y-up** (X+ droite, Y+ haut, Z+ avant), similaire à celui de Minecraft et plus naturel pour la navigation en scène 3D.

Toutefois, une erreur s'est glissée lors de la refactorisation : une simple rotation à 180° avec permutation de Y et Z ne suffit pas, il faut également inverser X. L'axe X s'est donc retrouvé inversé (X+ gauche au lieu de X+ droite), produisant un repère légèrement non standard.



Informations ajoutées au JSON

Le JSON possède une information en plus : une liste de positions où il y a des arbres.

Difficultés

Problèmes d'équipe

Le niveau technique étant hétérogène au sein du binôme, la répartition des tâches a été déséquilibrée, une grande partie du travail ayant été prise en charge par un seul membre. Ce déséquilibre, couplé à un manque d'anticipation, d'organisation et d'investissement de la part de l'autre membre, a généré des tensions et ralenti la progression globale du projet.

Problèmes du sujet

Le projet de base présentait de nombreux problèmes non documentés, comme les accès invalides aux VBO ou les chemins de shaders en relatif, rendant l'identification des bugs particulièrement difficile. De plus, la charge

de travail est nettement supérieure aux autres projets du semestre, d'autant que les TD associés à cette matière ne disposaient d'aucune correction, ce qui a limité les possibilités d'apprentissage en amont.

Conclusion

Ce projet a été particulièrement difficile à mener à terme. Le sujet manquait de clarté sur certains points et différait de ce à quoi nous étions habitués. Le déséquilibre d'organisation et de compétences au sein du binôme a constitué une difficulté supplémentaire tout au long du projet.